

Concurrency... is that the
same as *Parallelism*?!

A tale of two threads...
Never a dull moment! 😊

How did the week go?!

- What did you learn?
- What are you curious about?
- ***I LOVE YOUR POSTINGS!***
- To me, the mapreduce thread was a little more interesting because it wasn't just like, but also DISLIKE!
- Let's build on that energy a little! More like a critique...
 - In both the postings and the interviews!

HOW do we learn all of this?!

- Gently!?!?
 - Everyone has a different starting point, but we will get it!
- Make it curiosity driven...
 - This really is programming by copying instructions step by step
 - That is a bit DULL?!
 - But if you turn it around and say, “hey, what is *REALLY* GOING ON HERE?!”
 - YOU WILL TAP INTO YOUR SUPERPOWER!!!! 😊
- That rubric is quite big?!
 - Your code, your demo, your answering of questions...
 - Your ASKING questions, helping others, teamwork makes the dreamwork!

Where are we!?

- Amdahl's Law
- CAP theorem (simplification of a deep topic!)
- AWS gymnastics (not super intellectually stimulating but FUN!?)
 - You can use Oracle Cloud Infrastructure (OCI)
 - Java? Go? Or Rust! Or Pthreads!
- A little more on concepts about concurrency, TODAY!

Tons of tutorials out there

- Pthreads and a lot of mechanism for C and C++
 - Semaphores!
- Channels in Go
- Mutexes in Rust
- In Java
 - Synchronized in Java 😊
 - <https://docs.oracle.com/javase/tutorial/>
 - <https://docs.oracle.com/javase/tutorial/essential/concurrency/index.html>
- Please share others!!!

YAY?!

- What is GREAT about *concurrency*?
 - Lets think of this in terms of the applications you are running on your laptop...
 - Thank goodness we can run more than 1 at a time?!?! 😊
 - You might want to take the Systems course!
 - Just because we have support for concurrency, doesn't mean we really have parallelism!
- What is GREAT about *parallelism*?
 - WOAHH, now we are TALKING!
 - When we have designed things to run concurrently, they CAN run in parallel!
 - THAT *could* mean we get speedup for those bottlenecks!
 - But don't forget Amdahl's law!
 - Where are the bottlenecks

SOME problems are *embarrassingly parallelizable*

- Map/Reduce (GREAT thoughts on Piazza!)
- Some *serialized* behaviour, (Amdahl's law) but does well!
- Can be a little tricky to get right...
 - Data Skew
 - Breaking the data set down into “chunks”
 - Not every chunk takes the same amount of time to execute

The big picture... stragglers?!

Fork?!

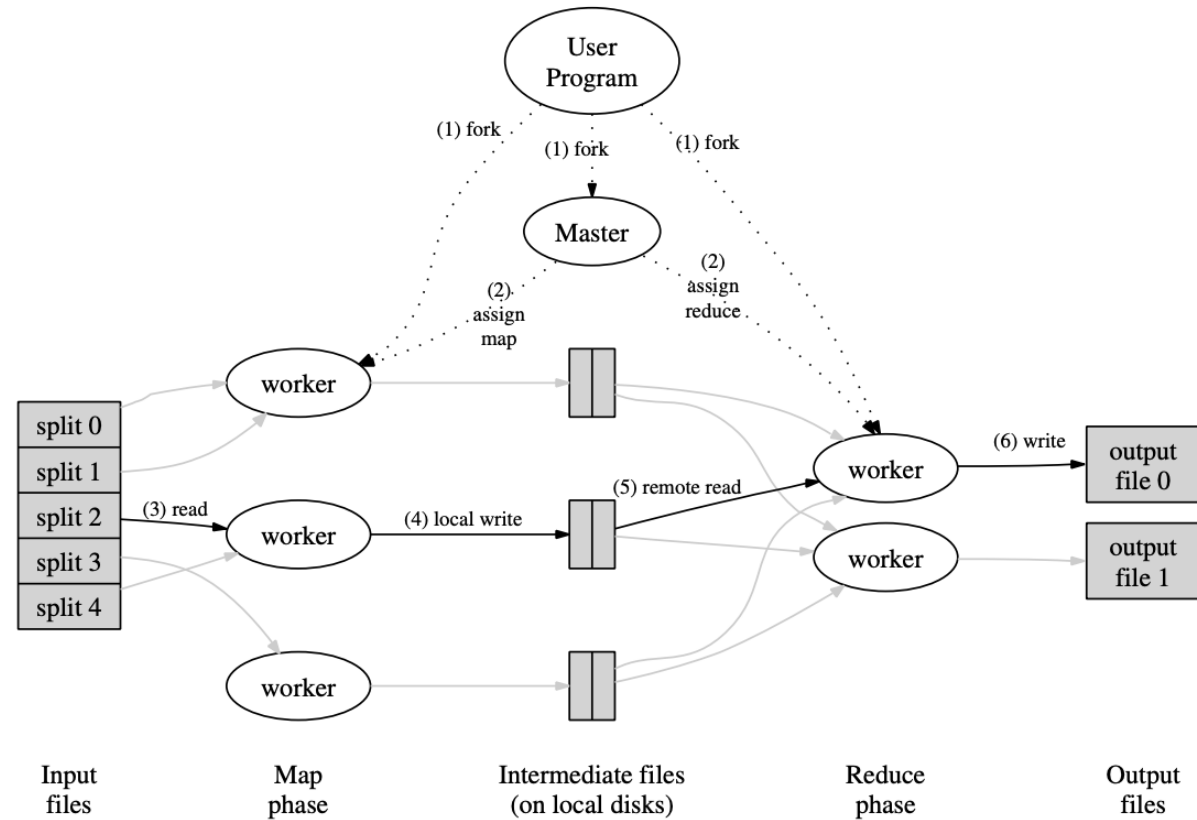
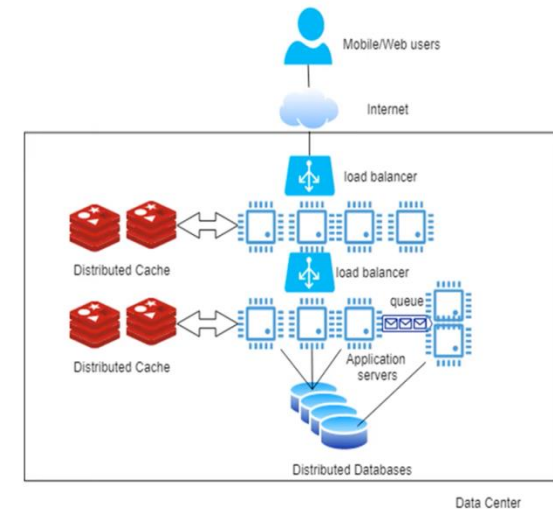
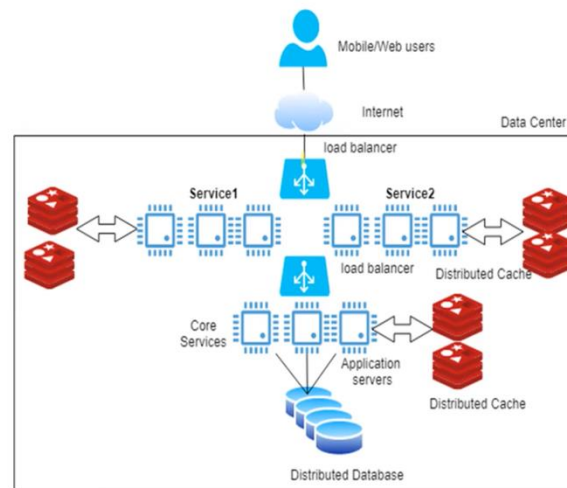
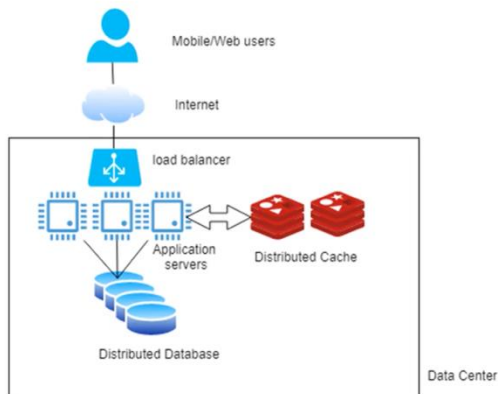
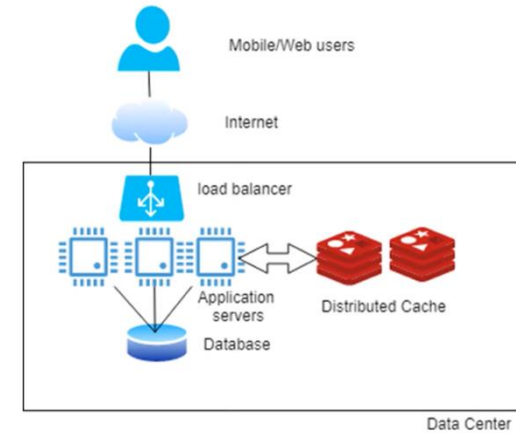
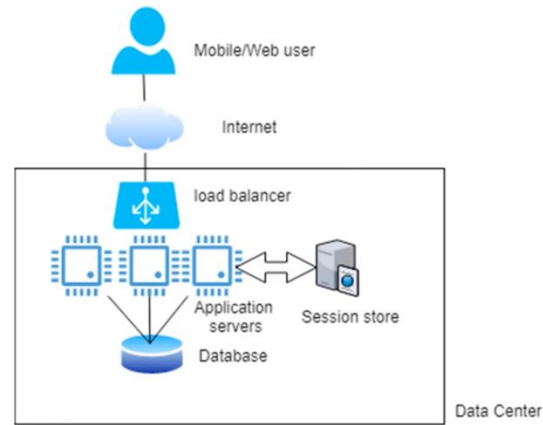
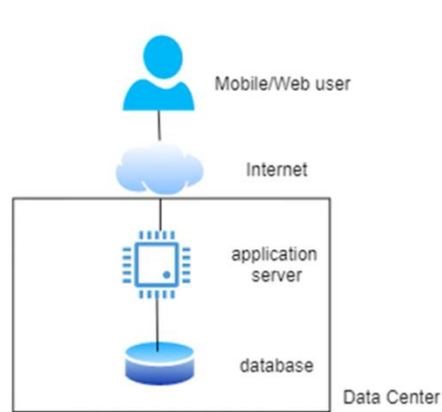
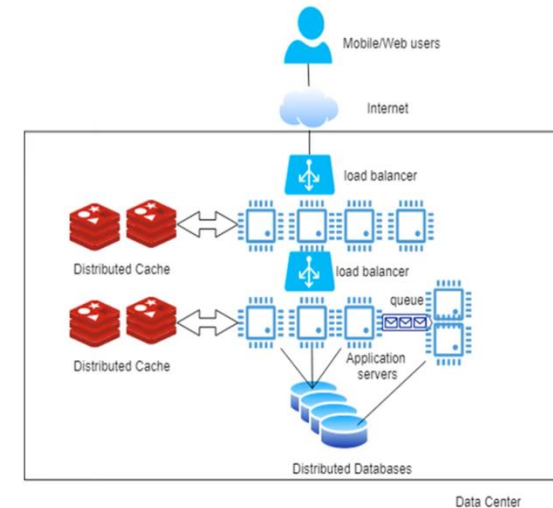
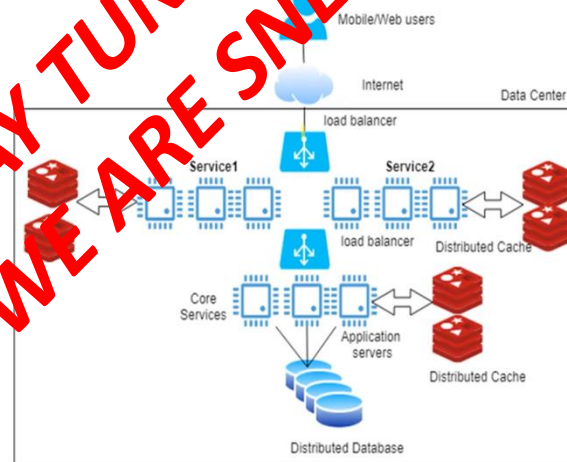
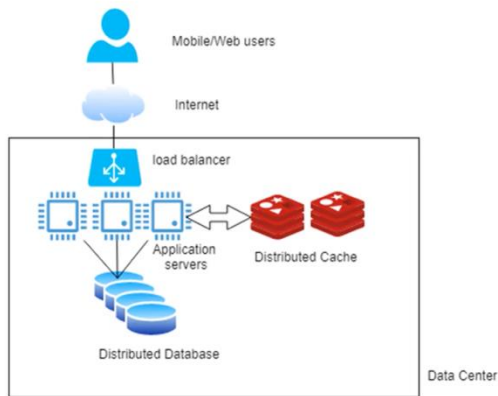
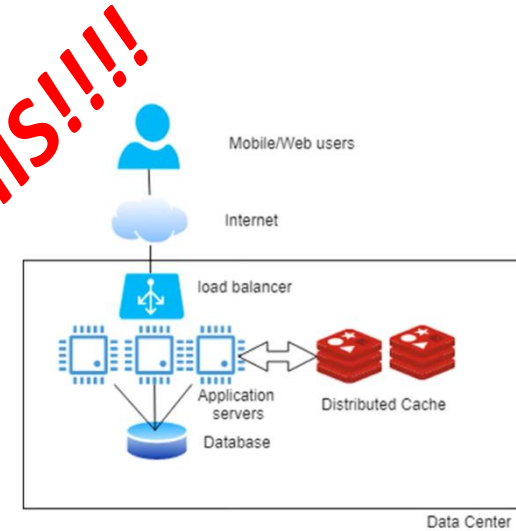
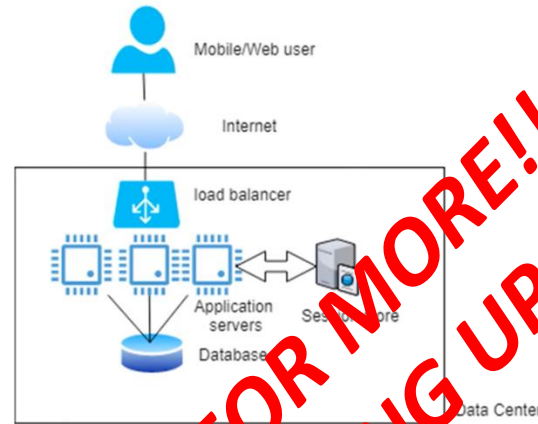
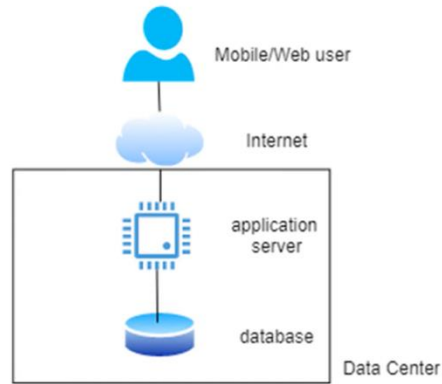


Figure 1: Execution overview

Scalability: tell the story!



Scalability: coming soon!



Why should we talk about Threads?

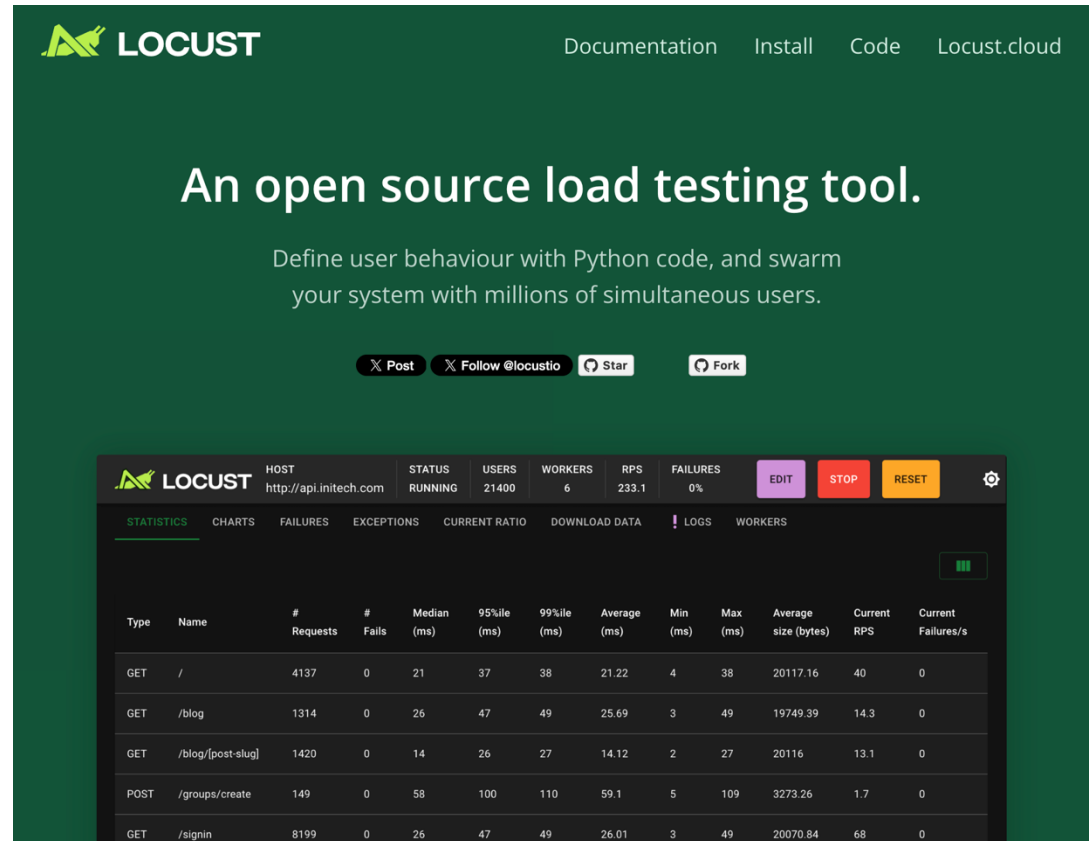
- Distributed systems are inherently concurrent, and we WANT parallelism!
 - Events happen on different nodes *at the same time*
 - *Unpredictable* order of events
- Threaded Code and Distributed Systems have the same *kinds* of bugs
- For web service, parallelism needed on each node to provide
 - Responsiveness to requests
- Data processing (video, LLM, scientific)
 - Do more calculations per second

Ways to think about the “landscape” (and costs!)

- Threads (might not have used this much yet!)
- Processes (your application running, “fork”)
- Containers (your docker container)
- Virtual Machines (your EC2 instance)

Important note!

- How do we generate a BLAST of requests for our backend?!
 - We have to do more than one *curl* at a time 😊
- Phone friends?
- Spawn processes?
- RUN THREADS!!!!



The screenshot displays the Locust web interface. At the top, the Locust logo is on the left, and links for Documentation, Install, Code, and Locust.cloud are on the right. The main heading reads "An open source load testing tool." followed by the description "Define user behaviour with Python code, and swarm your system with millions of simultaneous users." Below this are social media links for Post, Follow @locustio, Star, and Fork.

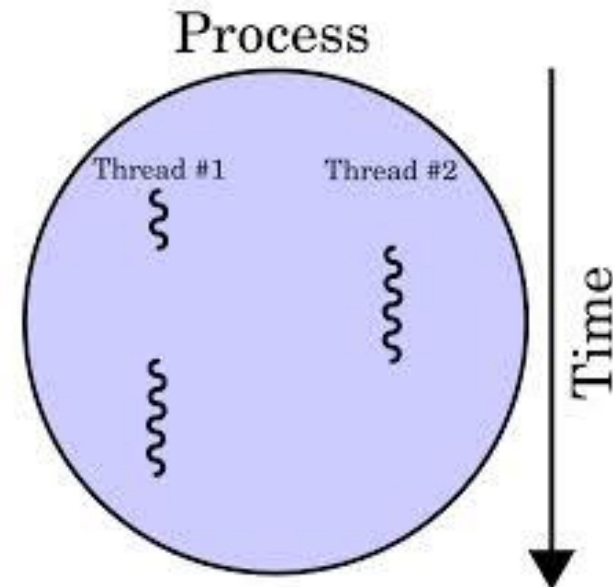
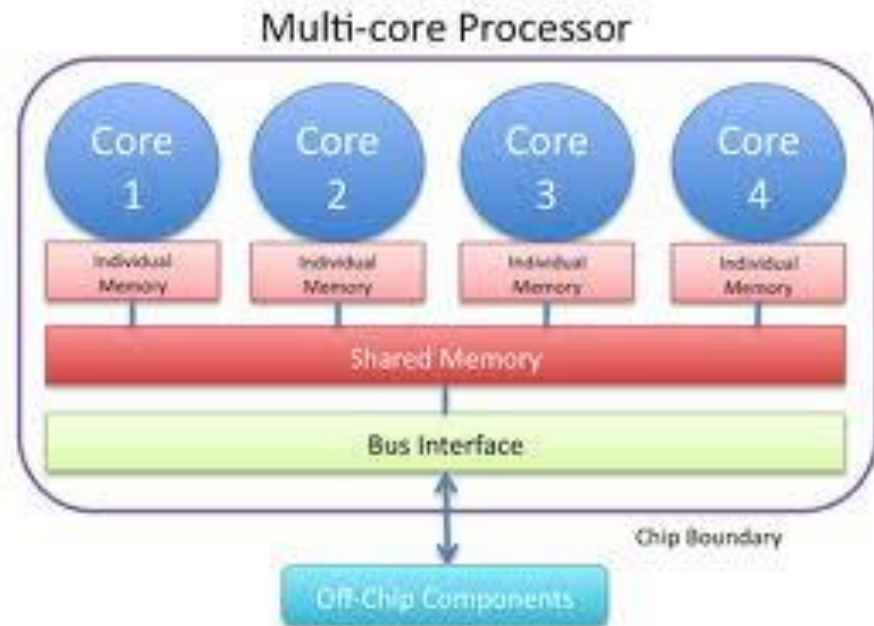
The interface shows a running test for the host `http://api.initech.com`. The status is "RUNNING" with 21400 users, 6 workers, and 233.1 RPS. There are 0 failures. Buttons for EDIT, STOP, and RESET are visible.

The "STATISTICS" tab is active, showing a table of request metrics:

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/	4137	0	21	37	38	21.22	4	38	20117.16	40	0
GET	/blog	1314	0	26	47	49	25.69	3	49	19749.39	14.3	0
GET	/blog/[post-slug]	1420	0	14	26	27	14.12	2	27	20116	13.1	0
POST	/groups/create	149	0	58	100	110	59.1	5	109	3273.26	1.7	0
GET	/signin	8199	0	26	47	49	26.01	3	49	20070.84	68	0

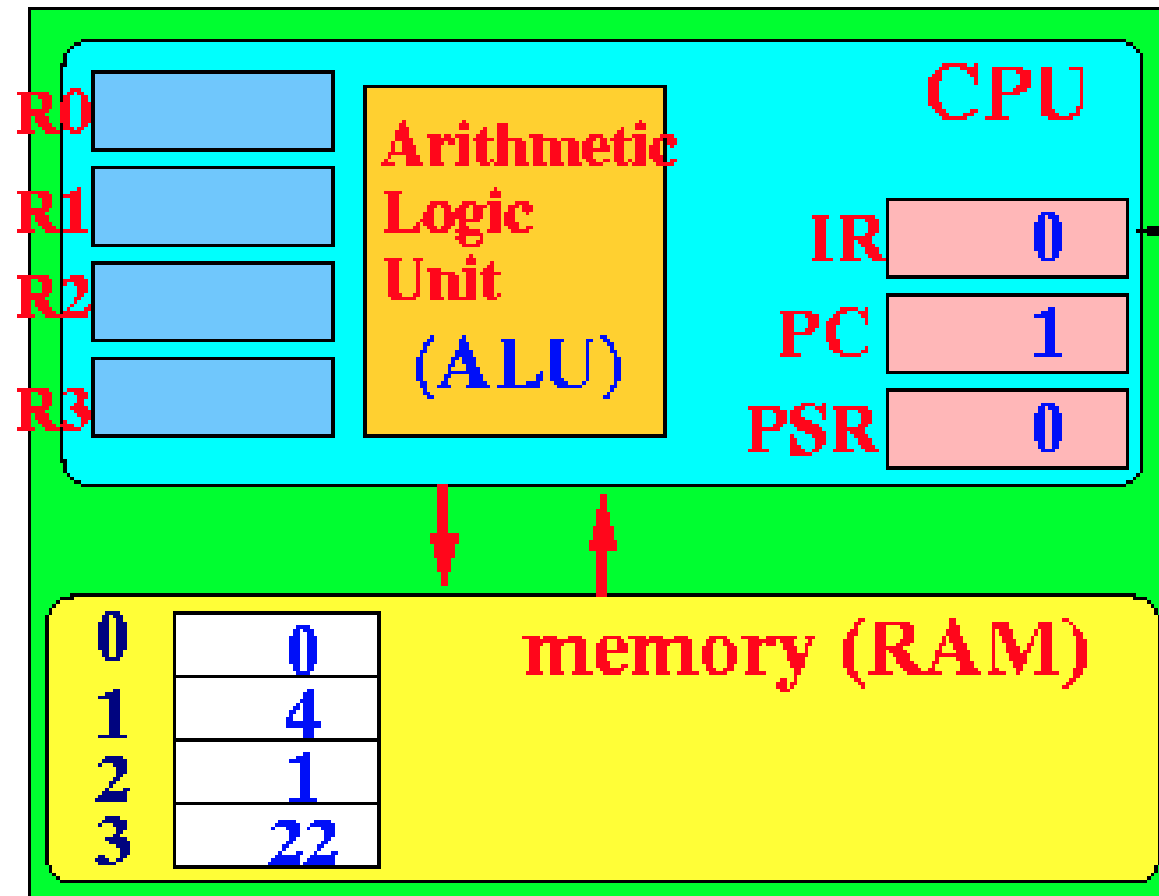
Why Threads and Concurrency

- Concurrent execution is necessary in many systems
 - Natural solution to many problems
 - Increase performance, e.g. do work while waiting for disk accesses
 - Necessary to exploit **multicore** – Physical limits to CPU clock speed
 - Moore's law!

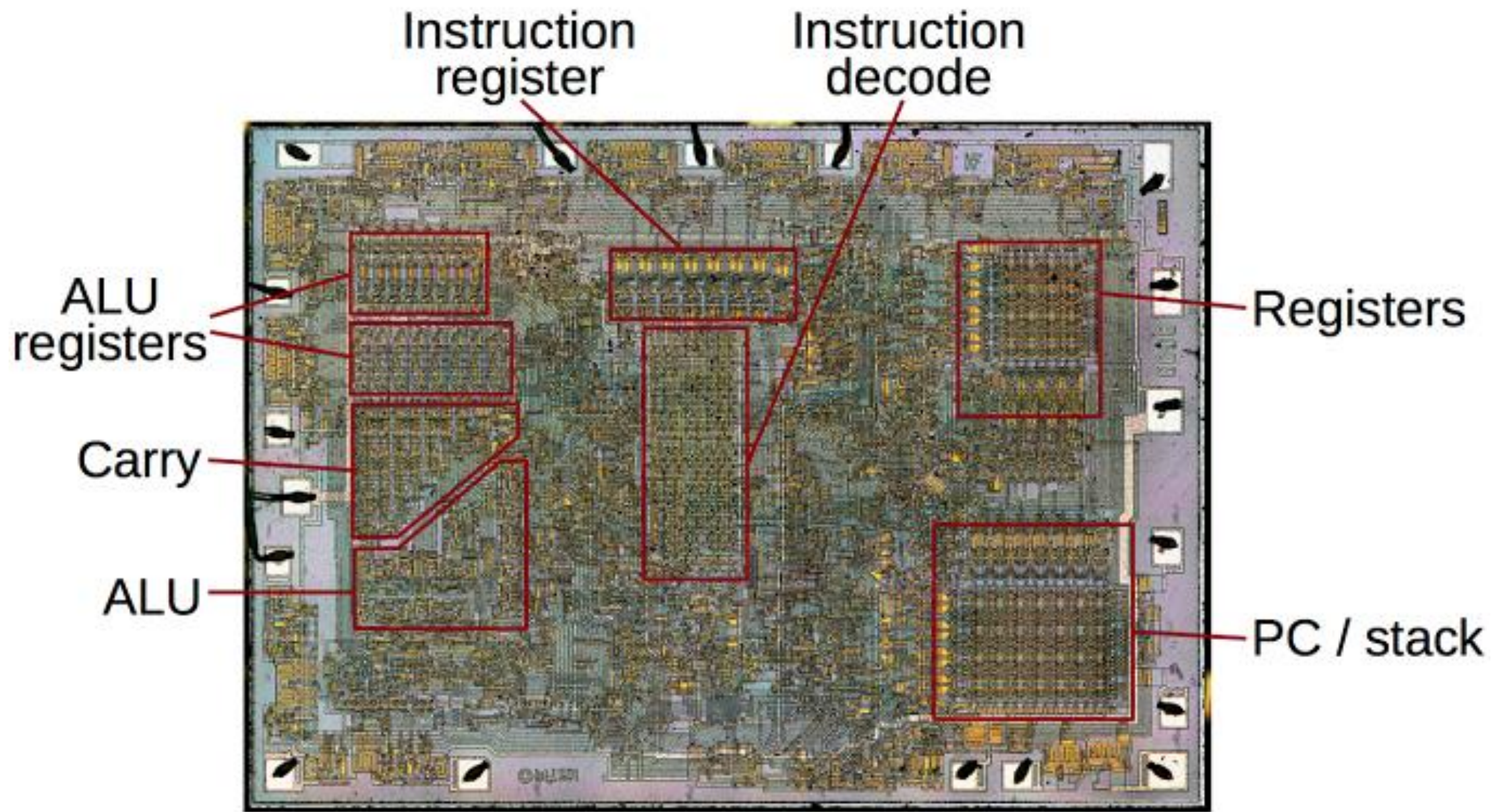


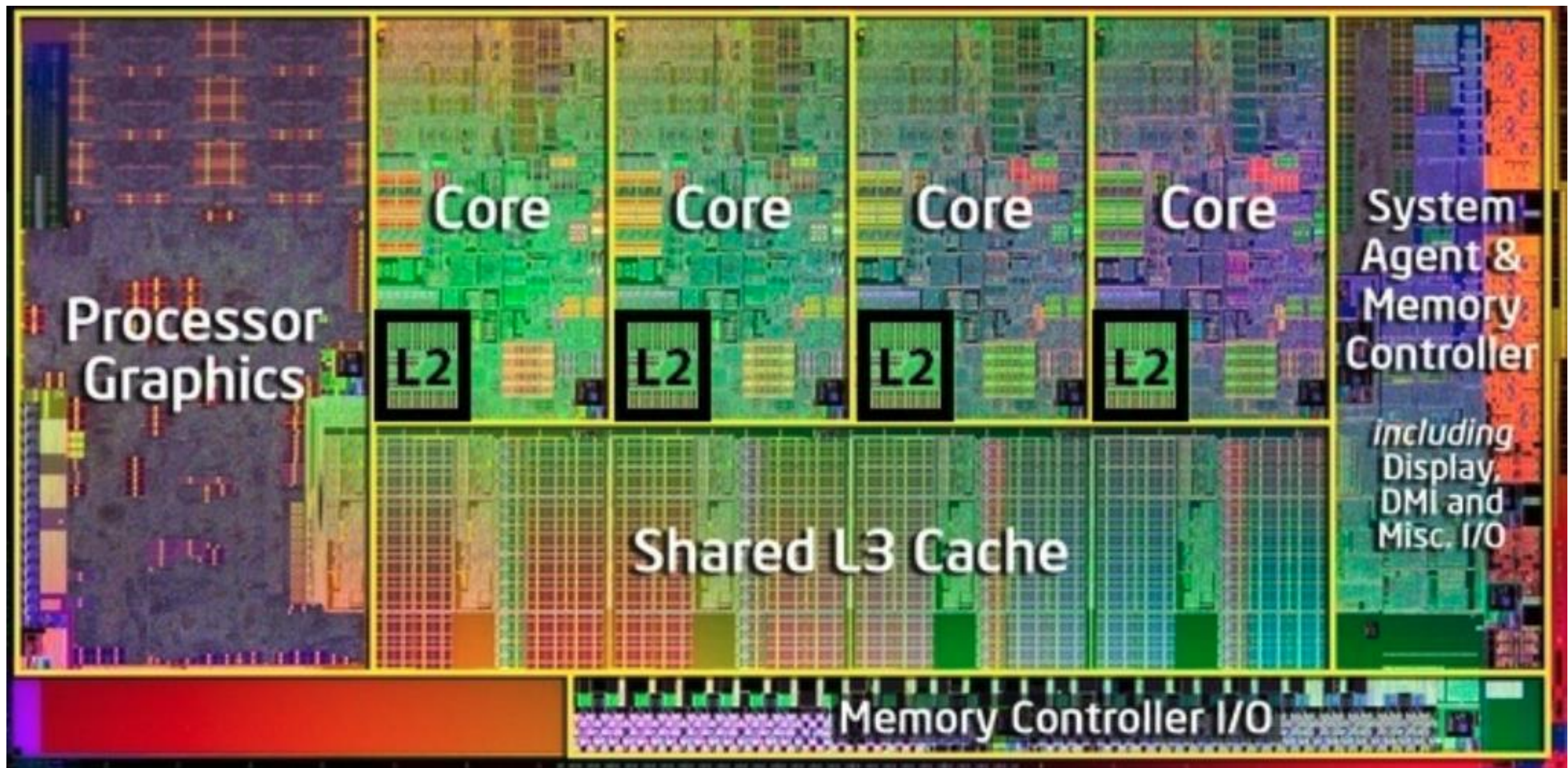
YESSSSSS!!!!

Computer system



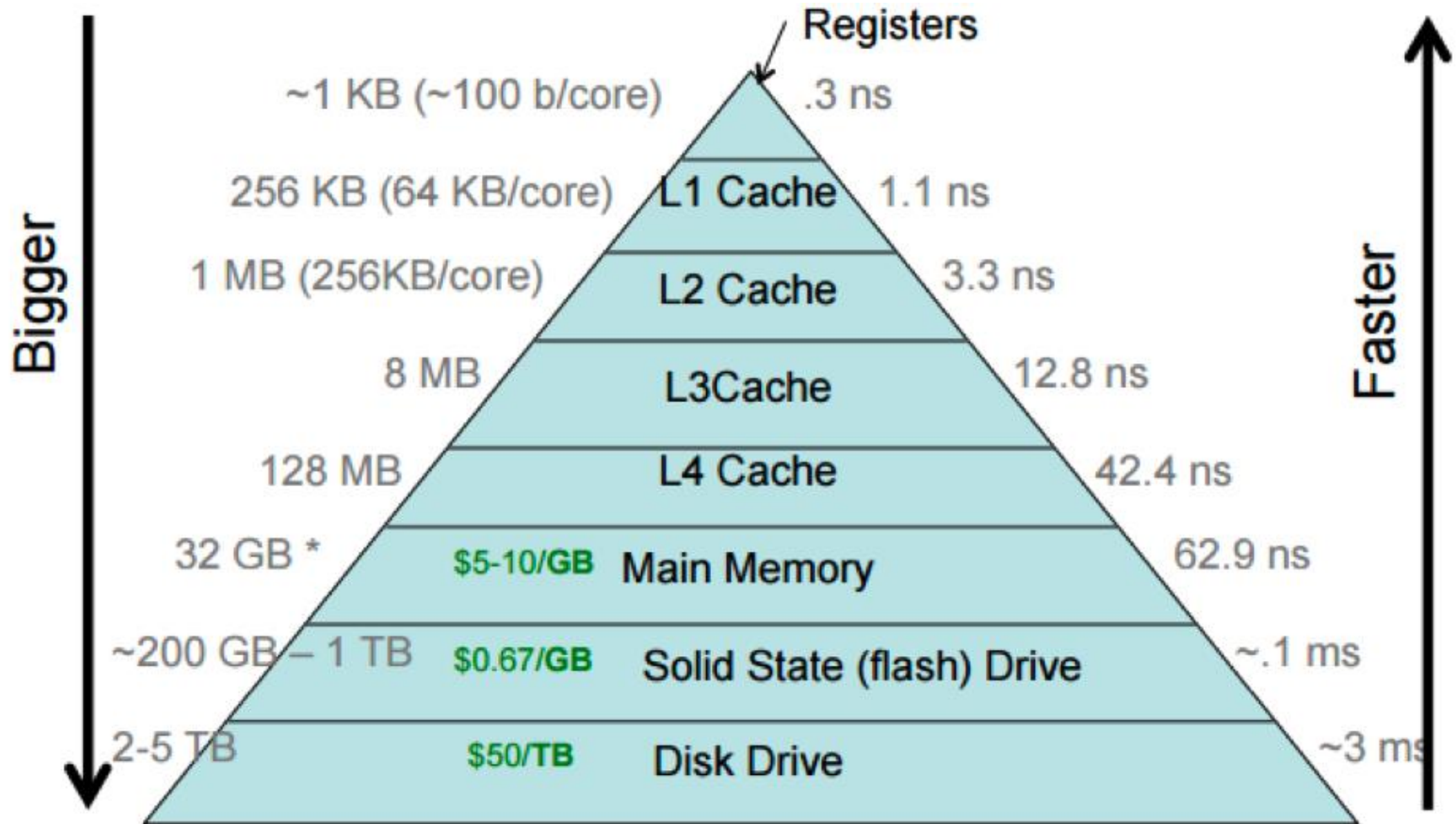
Instruction Register





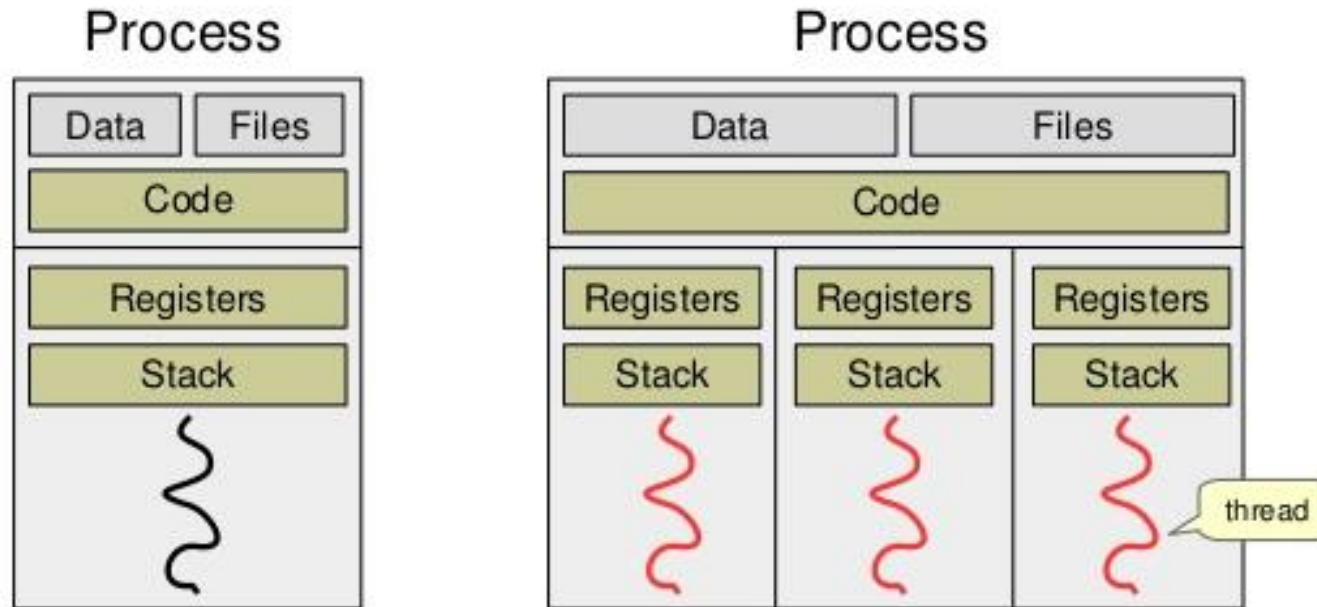
WAIT...

- Caches?
- Don't we use caches in distributed systems *too*?!
- YES, but these are different, they are local to the hardware on just one node...
- BUT THEY DO INTRODUCE COPIES, AND COPIES NEED TO BE KEPT CONSISTENT WITH EACH OTHER!



Threads vs Processes: Address Space

What is an OS process and thread?



Threads share the same address space

→ consumes less memory

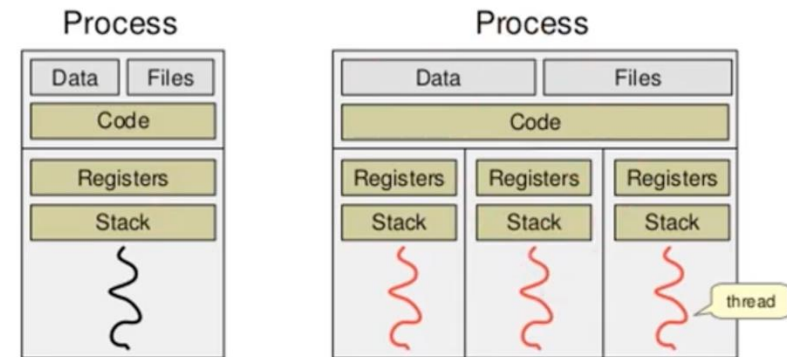
→ spawning is cheaper

→ context switching is cheaper

Thread versus process?

- Every process has at least one thread!
- It's possible to have more...
 - Sharing memory (fast sharing!)
 - Can exploit *parallelism*!
 - Lighter weight than processes
 - Scheduling is nondeterministic!
- Simple Java threads example
 - <https://docs.oracle.com/javase/tutorial/essential/concurrency/simple.html>

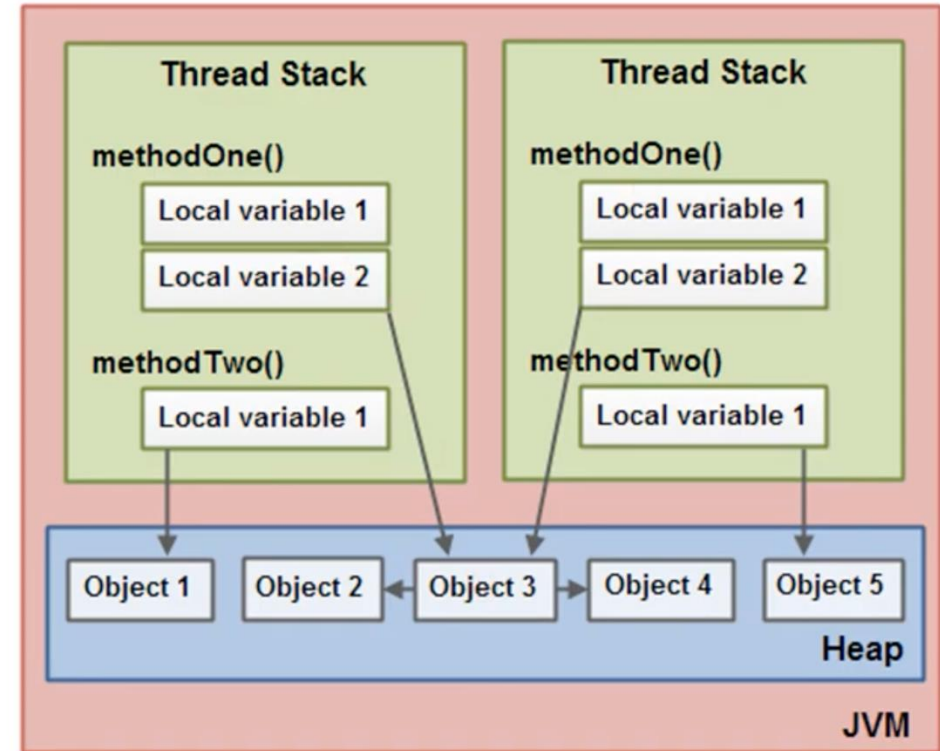
What is an OS process and thread?



Threads share the same address space
→ consumes less memory
→ spawning is cheaper
→ context switching is cheaper

Fundamental mechanisms

- Synchronization
- Coordination
- Thread pools
- Thread safe collections (phew!)
 - Libraries of data structures that are thread safe!
 - BUT THEY DO HAVE OVERHEADS!!!



Nondeterministic Behaviour means...

- Sharing is *NOT* caring!
- Every time you run your program it could output something different!
 - If threads share data
 - Assume thread 1 does: $x = x + 6$
 - Assume thread 2 does: $x = x + 1$
- Try it!
 - Can you make a RACE CONDITION happen in Java?
 - How do you prevent this?

```
class Counter {  
    private int c = 0;  
  
    public void increment() {  
        c++;  
    }  
  
    public void decrement() {  
        c--;  
    }  
  
    public int value() {  
        return c;  
    }  
}
```

Thread 1	Thread 2
Reads (x) into register	
Register value + 6	
Writes register value to (x)	
	Reads (x) into register
	Register value + 1
	Writes register value to (x)

Thread 1	Thread 2
Reads (x) into register	
Register value + 6	
	Reads (x) into register
	Register value + 1
	Writes register value to (x)
Writes register value to (x)	

LOCKS! AKA mutexes, semaphores...

- YAY!
- Imposes ordering
 - shared variables can be made SAFE!
- ONE thread at a time
 - **Serialize** the access to the shared object
 - Locks other threads out!
- *Synchronize* keyword in Java!
 - Creates a critical section
 - Mutually exclusive
 - ONE thread at a time, others block!
 - Known as a *Monitor lock*

```
public class SynchronizedCounter {  
    private int c = 0;  
  
    public synchronized void increment() {  
        c++;  
    }  
  
    public synchronized void decrement() {  
        c--;  
    }  
  
    public synchronized int value() {  
        return c;  
    }  
}
```

```
synchronized public void inc() {  
    count++;  
}
```


What about coordination?

- Dining Philosophers
- Readers/Writers
- Producer/Consumer

- Lots of examples out there!!!
 - Languages and Mechanisms are evolving!
- One for Producer/Consumer here
 - <https://docs.oracle.com/javase/tutorial/displayCode.html?code=https://docs.oracle.com/javase/tutorial/essential/concurrency/examples/ProducerConsumerExample.java>

Concurrent collections

- You can use *threadsafe* collections
 - Do that!
 - Less error prone than building your own
 - But this DOES introduce overheads...
 - Monitor locks
- Java.util.concurrent
 - <https://docs.oracle.com/javase/tutorial/essential/concurrency/collections.html>
 - These are great!
 - At what cost?!

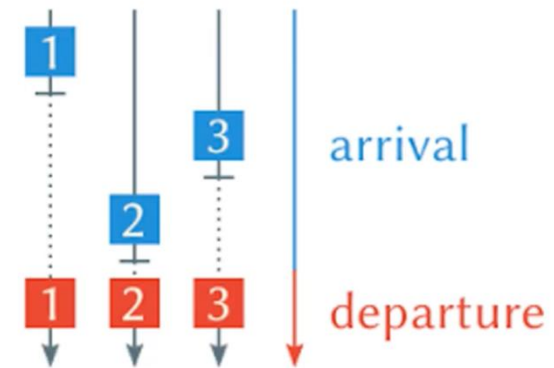
ANOTHER solution! Barrier synchronization

- A couple of ways to do it...
- Countdown Barrier
- Once we allllll hit zero we can GO!

- Initialize a count
 - `await()/countdown()`

- Example

<https://docs.oracle.com/javase/7/docs/api/java/util/concurrent/CountDownLatch.html>



KNOW there is much more!

- *Waitgroups* in Go!
 - Many examples
 - This one came from
 - <https://www.educative.io/edpresso/how-to-use-waitgroup-in-golang!>

```
1 package main
2
3 import (
4     "fmt"
5     "sync"
6 )
7
8
9 func main() {
10     //Goroutine execution 3
11     var wg sync.WaitGroup
12     wg.Add(2)
13     go func(){
14         count("sheep")
15         wg.Done()
16     }()
17
18     go func(){
19         count("cow")
20         wg.Done()
21     }()
22     wg.Wait()
23 }
24 func count(thing string){
25     for i:=0; i<=3; i++){
26         fmt.Println(thing)
27     }
28 }
```

Thread Efficiency

- Threads are lightweight compared to processes, they **share**
 - the same address space
 - Data and code
 - Share Security context
 - Environment variables
- Allocated their own stack space to support independent execution
- Context switching between threads is less expensive than between processes
- Cost of thread intercommunication is lower than process intercommunication

Threading a little different in every language?

- **Java** designed from the start with threads built-in to the language
- **C++** threading libraries are a little harder
- **Python** is easier than Java, but:
 - Multiprocessing package is an optional package, not built-in
 - Global Interpreter Lock (GIL) prevents concurrent updates
 - Real threading libraries are actually C/C++ plugins
- **GoLang** is arguably easier than Java but
 - It's a whole different model!!!

Simple Threads in Java (one way, extend Thread class!)

```
public class MyThread extends Thread {  
  
    public void run() {  
        System.out.println("MyThread  
running");  
    }  
}  
  
// and to create and start ...  
MyThread myThread = new MyThread();  
myThread.start();
```

```
public class SimpleThreadExample {  
  
    public static void main(String[] args) {  
  
        // create 3 threads and give them names  
        NamingRunnable name0 = new NamingRunnable("thread0");  
        NamingRunnable name1 = new NamingRunnable("thread1");  
        NamingRunnable name2 = new NamingRunnable("thread2");  
  
        //Create the threads  
        Thread t0 = new Thread(name0);  
        Thread t1 = new Thread(name1);  
        Thread t2 = new Thread(name2);  
  
        // start the threads  
        t0.start();           t1.start();           t2.start();  
        try { //delay the main thread for a second  
            Thread.currentThread().sleep(1000);  
        } catch (InterruptedException e) {}  
  
        //Display info about the main thread and terminate  
        System.out.println(Thread.currentThread());  
    }  
}
```


Simple Threads in Java

- **Wait for threads to terminate**

```
private static int NUMTHREADS = 10;
NamingRunnable[] runnables = new NamingRunnable[NUMTHREADS];

for (int i = 0; i < NUMTHREADS; i++) {
    runnables[i] = new NamingRunnable("thread:" + i);
}

Thread threads[] = new Thread[NUMTHREADS];
for (int i=0; i < NUMTHREADS ; i++ ) {
    threads[i] = new Thread (runnables[i]);
    threads[i].start();
}

// wait for each one to finish
try {
    for (int i=0; i < NUMTHREADS ; i++ ) {
        threads[i].join();
    }
} catch (InterruptedException e) {}

System.out.println(Thread.currentThread());
```

Problems with Threads

- Concurrency makes things challenging
 - Once we REALLY get parallelism...
 - Race conditions
 - Deadlocks
 - Source of problems
 - Non-determinism
 - Interleavings: order of Thread execution

Root cause: Non-determinism

- Sequential programs exhibit deterministic behavior
 - Thread 1 { a=2; b=a+6; }
 - Thread 2 { x=9; y=x-3; }
 - Whatever order the scheduler runs these threads in, the result will always be the same
 - No shared variables
- Race conditions are caused by **non-deterministic** behavior
 - WE HAVE NO CONTROL OVER THE PARELLISM!!!!
 - This means there may be interleavings that result in a WEIRD output!!!

Example of a Race Condition

```
public class RequestCounterBad {
    private static final int NUM_THREADS = 100000;
    private int count = 0;

    public void inc() {
        count++;
    }

    public int getVal() {
        return this.count;
    }

    public static void main(String[] args) throws InterruptedException {
        final RequestCounterBad counter = new RequestCounterBad();

        for (int i = 0; i < NUM_THREADS; i++) {
            Runnable thread = () -> { counter.inc(); };
            new Thread(thread).start();
        }

        Thread.sleep(5000);
        System.out.println("Value should be " + NUM_THREADS + " - It is " + counter.getVal());
    }
}
```

The results:

- Value should be equal to 100000 it is: 99988
- Value should be equal to 100000 it is: 99989

Removing Race Conditions – Locks

- Use locks to impose ordering constraints
 - Lock shared variables so they can be accessed only by a single thread at once
 - Serialized access to shared resources
 - "Serialize" - order into a single, non-concurrent execution
- Each Thread wishing to access a variable:
 - *gets* the lock
 - *changes* the variable
 - *releases* the lock
- If the lock is set, all other threads wait for it to be released
 - Which thread proceeds next?
 - Locks sometimes known as **semaphores**

Semaphore

The inventor (Dijkstra) was Dutch and used Dutch words



Java synchronization features

```
public class SynchronizedCounter {  
    private int counter = 0;  
    public synchronized void increment() { counter++; }  
    public synchronized void decrement() { counter--; }  
}
```

- `Synchronized` : only one thread can execute this method at a time. Others will be blocked
 - Can only have one train on one section of track at a time
- Known as **critical section**
 - Invocations of a synchronized methods cannot overlap
 - less error-prone than manual locks as release is automatic

Monitor Locks

- *Synchronization* is implemented using *monitors*.
- Each object in Java has its own monitor, which a thread can *lock* or *unlock*.
- Only one thread at a time may hold a lock on a monitor.
 - `synchronized (this) ;`
 - `synchronized (Object) ;`
- Any other threads attempting to lock that monitor are blocked until they can obtain a lock on that monitor.

Atomic Variables

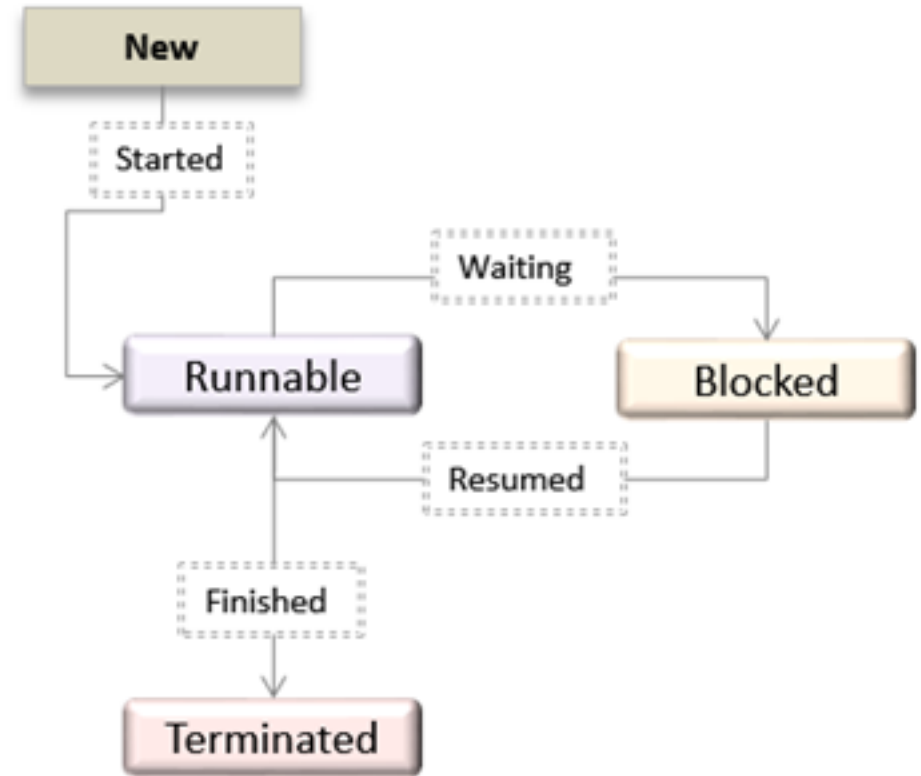
- For modifying Shared variables

```
public class SharedVariable {  
    private final AtomicInteger val= new AtomicInteger();  
  
    public void incrementNumber() {  
        val.getAndIncrement();  
    }  
  
    public int getNumber() { val.get(); }
```

- Everything else stays the same
- Implemented using monitors behind the scenes

Thread States

- New Thread state (Ready-to-run state)
 - Created but not started
- Runnable state (Running state)
 - Started and either running or waiting to run
- Blocked
- Dead state
 - Stop() called or run() terminates



Thread Priority

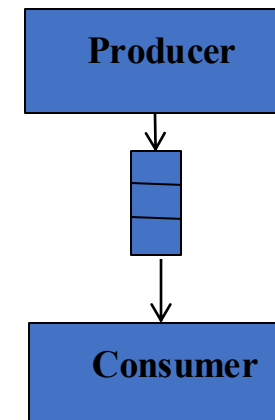
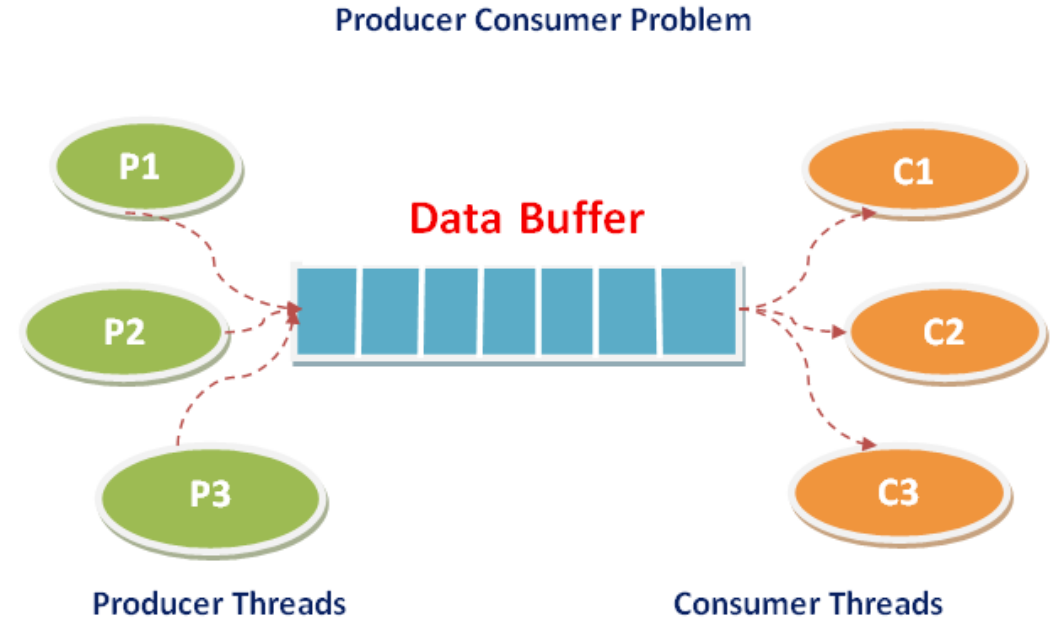
- In Java every thread has a priority
 - Higher priority threads get scheduled more frequently than lower priority threads
- A Java thread inherits its priority from its parent
 - MIN_PRIORITY (0) Lowest Priority
 - NORM_PRIORITY (5) Default Priority
 - MAX_PRIORITY (10) Highest Priority

Thread Scheduling

- The thread scheduler chooses the *Runnable* thread with the highest priority for execution
- When multiple threads to choose from, scheduler chooses one in a *round-robin fashion*. The chosen thread will run until
 - a higher priority thread becomes *Runnable* (Pre-emptive)
 - it *yields*, or its `run()` method exits
 - its time allotment has expired (time-slicing)

Thread Co-ordination

- Need to ensure
 - Consumer waits if buffer empty
 - Consumer can remove item when buffer not empty
 - Producer waits if buffer full
 - Producer can add item if buffer has free space
- Guards
 - Producer-Consumer style examples require 'guards'
 - Producer stores message in a shared buffer
 - Except when full
 - Consumers retrieve messages from buffer
 - Wait when empty



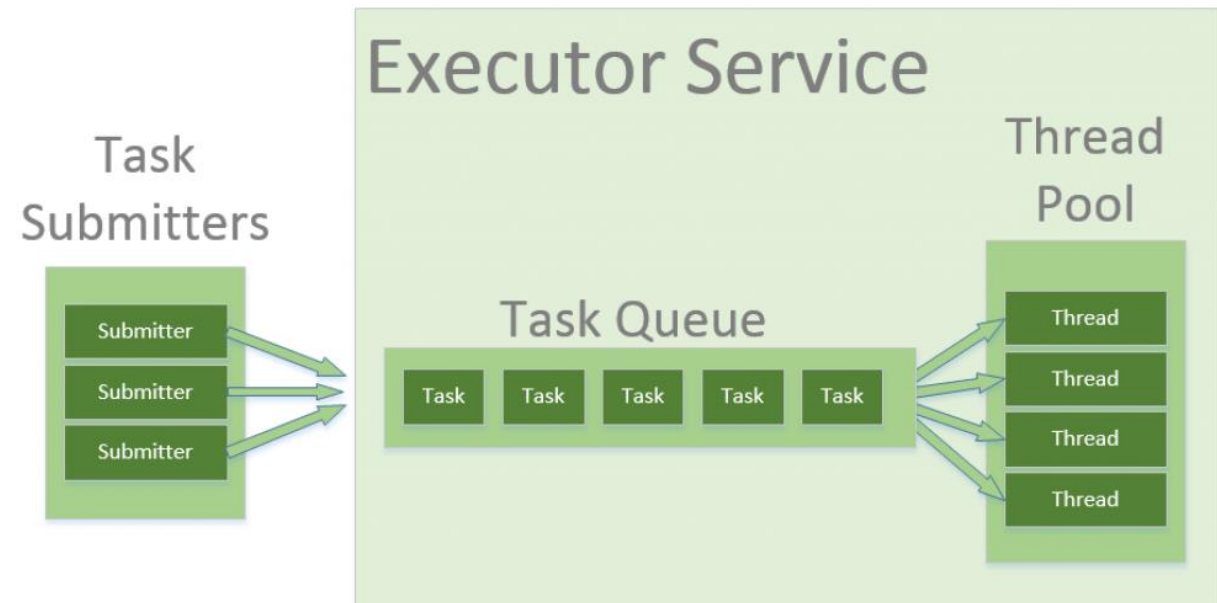
Java Guards

- `wait()` and `notify()` statements
- Wait and notify provide thread inter-communication that synchronizes on the same object.
 - `final void wait(long timeout)` throws `InterruptedException`
 - `final void wait()` throws `InterruptedException`
 - `final void notify()`
 - `final void notifyAll()`

Thread Pools!

- Threads consume resources
 - Creation
 - Context switching
 - Memory per Thread
- Thread pools provide a mechanism for an application to control number of threads
 - Reuse threads to minimize creation cost.
 - Fixed memory size
- Support a task-based programming model
- Executor Framework
 - An object that executes submitted Runnable tasks.
 - For example, rather than invoking `new Thread(new RunnableTask()).start()` for each of a set of tasks,

```
Executor executor = anExecutor;  
executor.execute(new RunnableTask1());  
executor.execute(new RunnableTask2());
```



Thread Summary

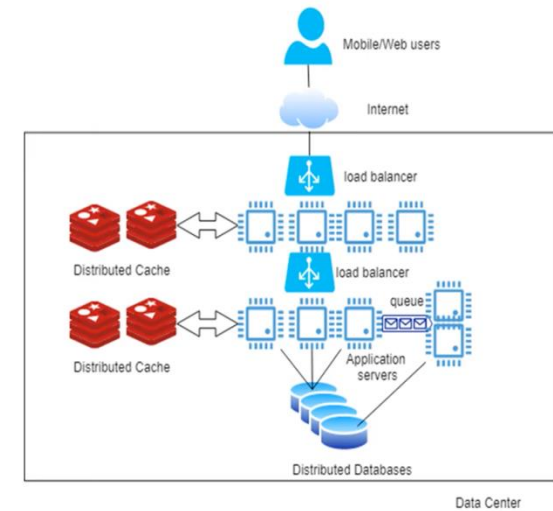
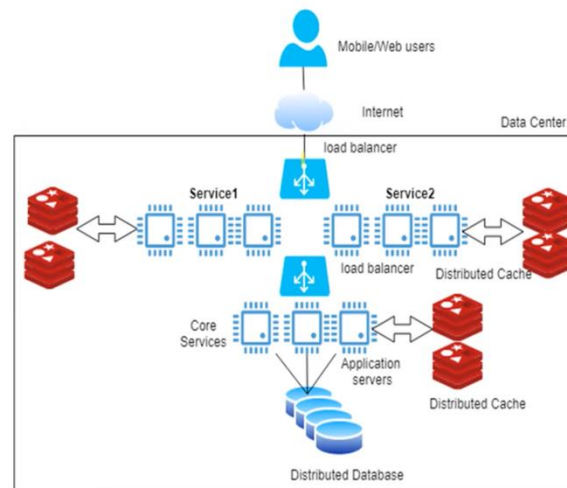
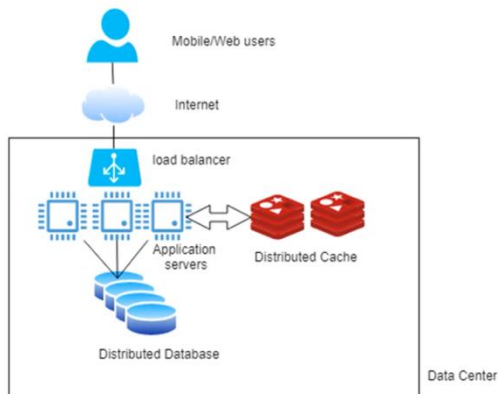
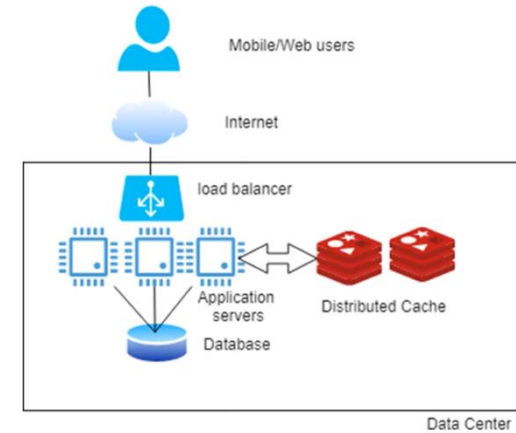
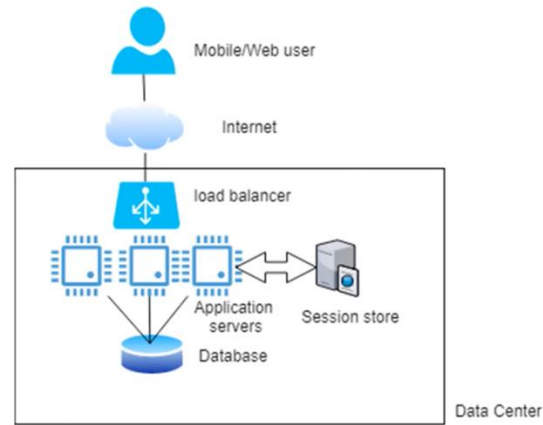
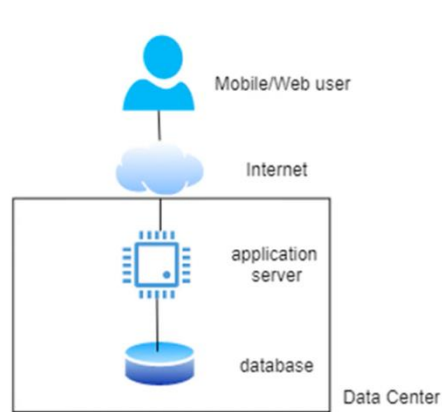
- Concurrency is fundamental to software systems!
 - We can thank Dijkstra for the multi-process model 😊
- Parallelism introduces problems of race conditions and deadlocks
- Synchronization required as a solution
- Threads move through various states during their lifetime
- Scheduler makes decisions on which thread to run based on their state and priority
- Executors and concurrent utility classes simplify threaded programs

- **Thread bugs are hard and typically occur during production!!!**
- **BUT WHY?!?!?!?!?!?!?!!**

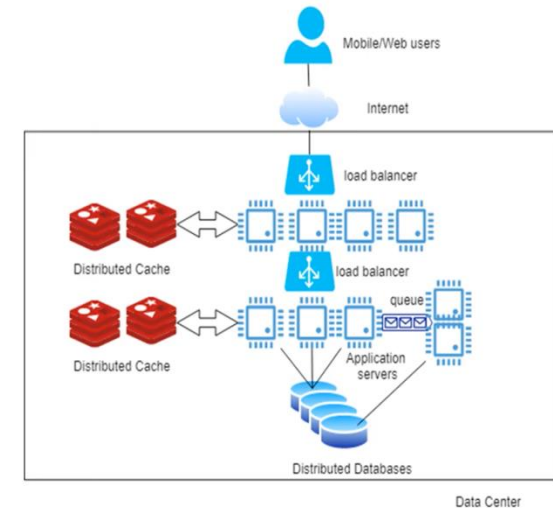
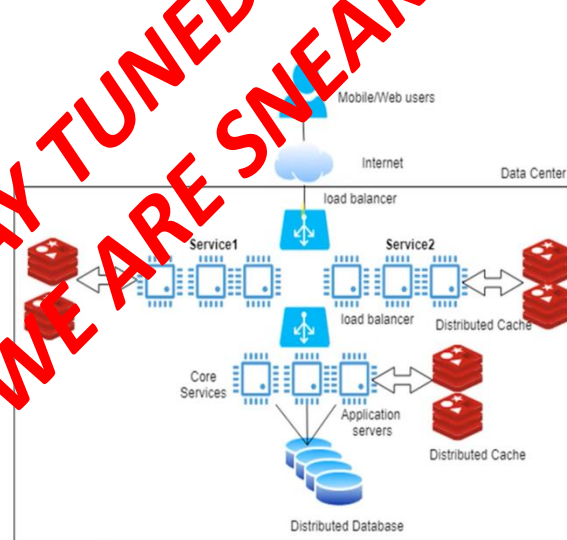
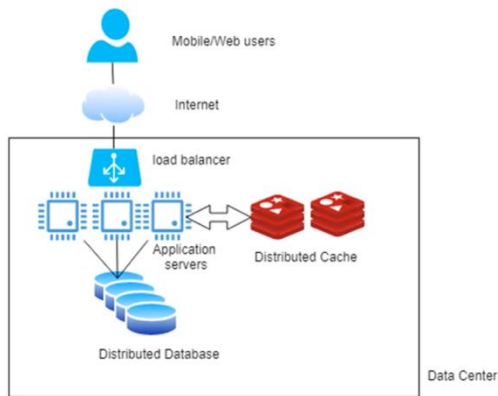
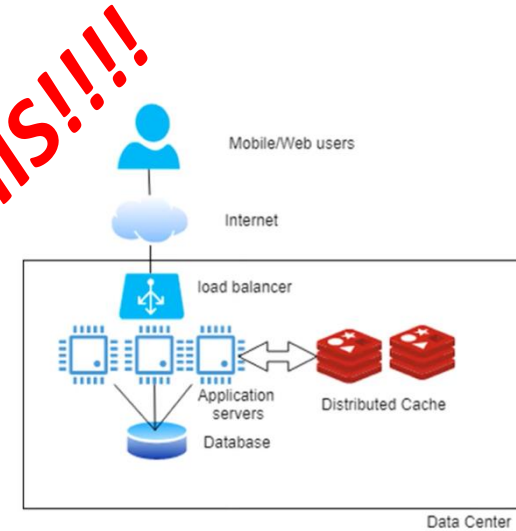
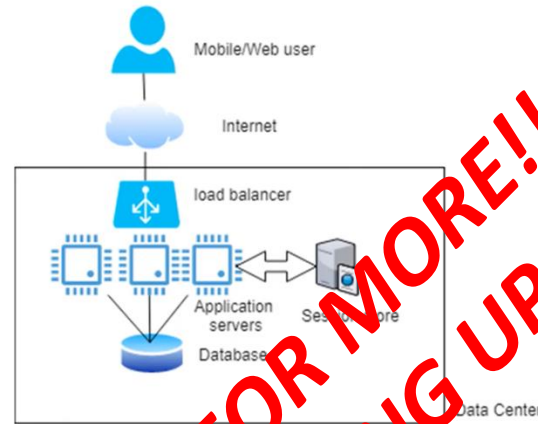
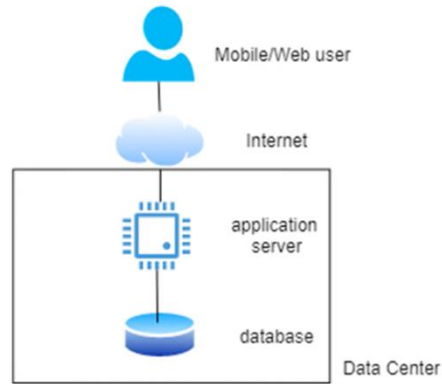
So WHAT?!

- If this is not the focus of the course, what are we learning here?
- Distributed systems don't share memory!
- How does any of this matter?
 - Well... we do share “resources” in distributed systems
 - And our whole goal is to be able to scale up and down easily!

Scalability: tell the story!

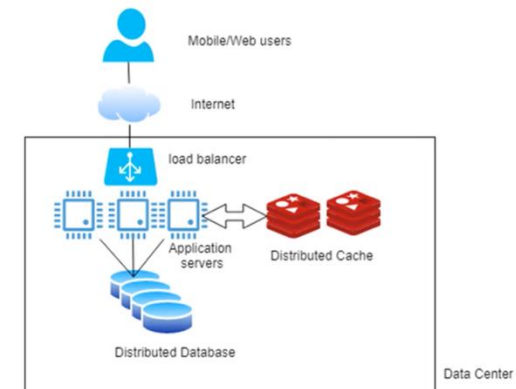
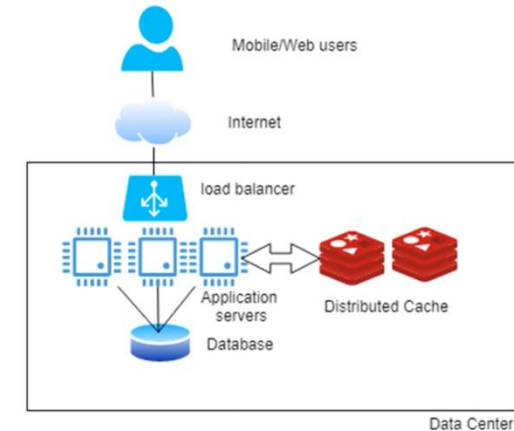


Scalability: coming soon!



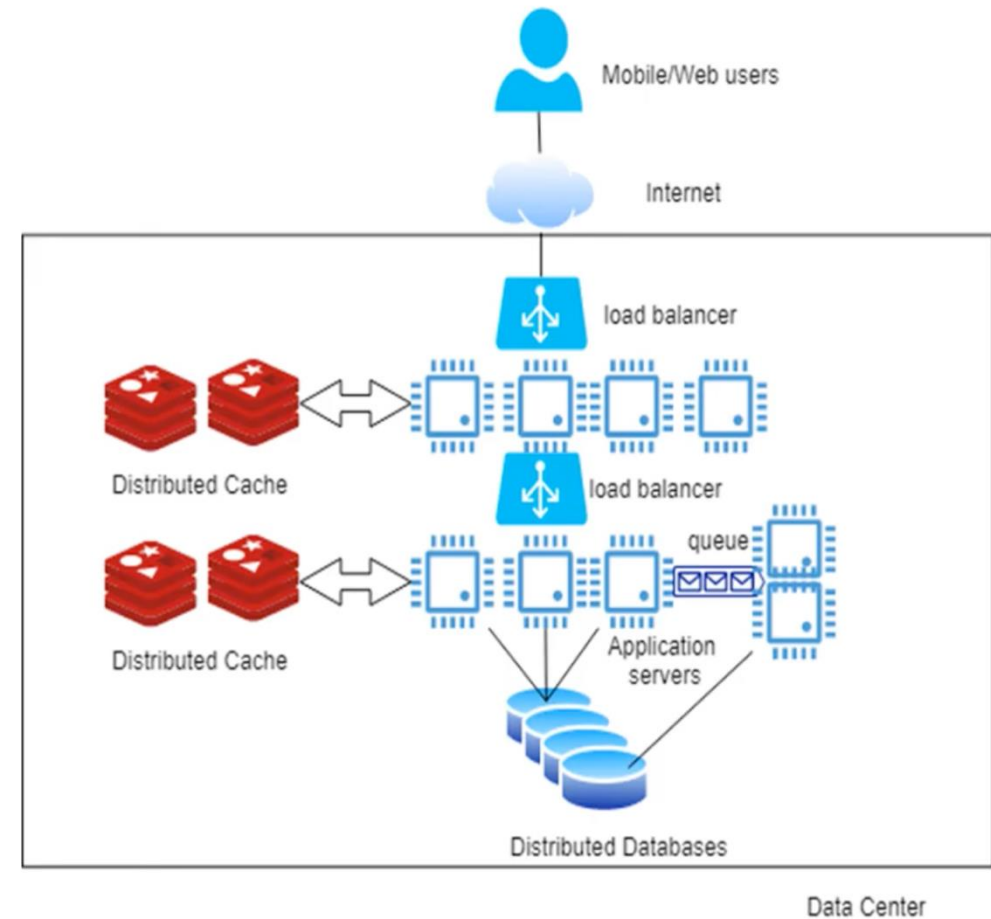
Scale UP versus Scale OUT?

- What is the first bottleneck?
 - Yup, the database!
 - Can **Scale Up** the database...
 - More Storage and RAM
 - *Still Single Point of Failure*
 - Can **Scale Out** the application servers
- Also use a distributed cache
 - Can be quickly retrieved
 - Distributed key-value stores
- AND if you need more, *distributed databases*!
 - SQL versus noSQL
 - Replication has tradeoffs
 - Cloud provider can handle the administration
 - dynamoDB



Eventual consistency

- Application servers use a queue
- Not waiting for the database!
- Request gets acknowledged
- Another service writes it out
- Data updated eventually
 - Server “promises” to do it!



Could be *ANY* Scalable application!

You work for the analytics department at [Bandcamp](#)

Bandcamp provides a listening platform for music, and collects data on it's listeners. Every time a listener starts playing a song, the time spent listening to the song and the listener ID are recorded.

In this course, through a series of homeworks and milestones, we'll build a scalable distributed cloud-based system that can handle data sets like this. This data can then be used as a basis for data analysis, answering such questions as:

- which songs are most heavily favoured?
- which listeners play the most songs?
- how many songs do listeners play on average per day from band X?
- handle payments!

Questions?